

Configuración de Redes Neuronales

Sí, absolutamente. De hecho, el diseño de la capa de salida, su función de activación y la función de coste a minimizar **dependen directamente del tipo de problema** que pretendas resolver.

Lo más interesante de cara a tu examen es que, si emparejas la función de activación correcta con su función de coste ideal, ocurre una especie de "magia matemática": las derivadas complejas de ambas funciones se cancelan mutuamente, lo que **simplifica drásticamente el cálculo del error en la capa de salida** (Δ^L) a la hora de hacer retropropagación a mano.

Estas son las tres configuraciones típicas o "de manual":

1. Tareas de Regresión (predecir valores numéricos)

- **Configuración:** La capa de salida tiene tantas neuronas como valores a predecir (ej. n neuronas). Su función de activación es la **función identidad** y se entrena minimizando el **Error Cuadrático Medio (MSE)**.
- **Cálculo facilitado:** El cálculo del error inicial que tienes que propagar hacia atrás (Δ^L) se reduce a una simple resta multiplicada por una constante: $\Delta^L = \frac{2}{n}(a^L - y)$.

2. Tareas de Clasificación Binaria (dos clases, 0 o 1)

- **Configuración:** La capa de salida tiene **1 sola neurona** artificial. Se utiliza la **función sigmoide** para que la salida esté entre 0 y 1, y la función de coste a minimizar es la **entropía cruzada binaria**.
- **Cálculo facilitado (Truco de examen):** Si usas esta pareja exacta, todo el horror de derivar logaritmos neperianos y exponenciales desaparece. El error en la capa de salida se simplifica directamente a $\Delta^L = a^L - y$. ¡Solo tienes que restar la respuesta correcta (y) de la predicción de la red (a^L)!

3. Tareas de Clasificación Multiclase (múltiples categorías)

- **Configuración:** Se usa una capa de salida con n neuronas (una por clase, codificando la salida en formato *one-hot*). La función de activación es la función **softmax** (que reparte las probabilidades de forma que sumen 1) y se usa la **entropía cruzada categórica** como función de coste.
- **Cálculo facilitado:** Al igual que en la clasificación binaria, si combinas *softmax* con entropía cruzada categórica, el vector de errores de la capa de salida vuelve a simplificarse de forma milagrosa a $\Delta^L = a^L - y$.

Además de la capa de salida, existe otra configuración típica para facilitar los cálculos en las **capas ocultas** (las intermedias). Hoy en día se recomienda utilizar casi siempre la función **rectificador (ReLU)** en estas capas. Para hacer los problemas a mano esto es una bendición, porque su derivada es inmediata: simplemente vale 1 si el valor de entrada era mayor que 0, y vale 0 si era menor o igual.

Conocer estas configuraciones teóricas no solo garantiza que la red neuronal funcione, sino que te ahorrará hacer derivadas en cadena larguísimas durante los ejercicios de examen.

Parejas de oro: Función de Activación y Función de Coste

Aquí tienes la tabla resumen con las "parejas de oro" para la capa de salida y las derivadas clave de las capas ocultas. Esta tabla te ahorrará muchísimo tiempo al calcular los vectores de error (Δ^L) en el algoritmo de retropropagación:

Tipo de Problema / Capa	Función de Activación	Función de Coste (Pérdida)	Cálculo Rápido del Error (Δ^L) o Derivada
Regresión (Valores continuos)	Identidad	Error Cuadrático Medio (MSE)	$\Delta^L = \frac{2}{n}(a^L - y)$
Clasificación Binaria	Sigmoide	Entropía Cruzada Binaria	$\Delta^L = a^L - y$
Clasificación Multiclase	Softmax	Entropía Cruzada Categórica	$\Delta^L = a^L - y$
Capas Ocultas (Moderna)	ReLU (Rectificador)	(Heredan el error de la capa siguiente)	Derivada: 1 si $z > 0$, 0 si $z \leq 0$
Capas Ocultas (Clásica)	Sigmoide	(Heredan el error de la capa siguiente)	Derivada: $a^l(1 - a^l)$

(Nota de examen: En la fórmula de regresión, la n representa la cantidad total de neuronas que tiene la capa de salida. Y recuerda siempre que, al aplicar los atajos $\Delta^L = a^L - y$, debes escribir a^L (la salida de la red) y y (la solución correcta) en formato de vector columna en vertical para poder hacer las restas elemento a elemento correctamente).

Diseño y entrenamiento de la red neuronal en tareas de clasificación

Para diseñar y entrenar adecuadamente una red neuronal en tareas de clasificación, es fundamental estructurar primero su arquitectura dependiendo del tipo de problema:

- **Para Clasificación Binaria:** Se utiliza **1 neurona en la capa de salida** con la función de activación **Sigmoide**. Las etiquetas reales se definen como 0 o 1, y la predicción final se obtiene estableciendo un umbral de corte (generalmente $u = 0.5$).
- **Para Clasificación Multiclase:** Se utilizan **tantas neuronas de salida como posibles clases**, aplicando una codificación vectorial *one-hot* para el objetivo. La función de activación es **Softmax**, que proporciona distribuciones de probabilidad, y la predicción final será la clase con mayor valor matemático (**arg max**). *(Nota: Para las capas ocultas intermedias, la teoría moderna recomienda utilizar de forma estandarizada la función de activación **Rectificador o ReLU**).*

Una vez configurada la arquitectura, las fórmulas matriciales y vectoriales que dirigen el entrenamiento durante una época son las siguientes:

1. Propagación hacia adelante (Forward)

Para que un ejemplo de entrada x transite por la red, se define la activación inicial como $a^1 = x$. A partir de ahí, para cada capa posterior ($l = 2, \dots, L$), se calcula la entrada neta ponderada (z^l) y la activación de las neuronas (a^l) mediante las ecuaciones:

$$z^l = W^l a^{l-1} + W_0^l$$

$$a^l = g^l(z^l)$$

(Donde W^l es la matriz de pesos de la capa, W_0^l es el vector de sesgos, y g^l es la función de activación aplicada a cada elemento).

2. Cálculo del Error

Para estimar la pérdida cometida por la red en una predicción frente al objetivo real y , se aplican las funciones de entropía:

- **Si es binaria (Entropía Cruzada Binaria):** $C = -y \log_e(a^L) - (1 - y) \log_e(1 - a^L)$.
- **Si es multiclase (Entropía Cruzada Categórica):** $C = -\sum_{k=1}^n y_k \log_e(a_k^L)$.

3. Propagación hacia atrás del error (Backpropagation)

Consiste en calcular un vector de error local o "Delta" (Δ^l) para cada capa, comenzando desde el final y retrocediendo hacia la entrada.

- **Error en la Capa de Salida (Δ^L):** El diseño de la red neuronal produce un atajo matemático vital. Al combinar matemáticamente la función Sigmoides con la Entropía Binaria (o la Softmax con la Entropía Categórica), la derivada del error se simplifica en una simple resta entre la predicción y el valor real:

$$\Delta^L = a^L - y$$

- **Error en las Capas Ocultas (Δ^l):** El error se propaga hacia el interior de la red multiplicando el Delta de la capa superior por la matriz de pesos transpuesta, y aplicando el producto de Hadamard ($\odot$) con la derivada de la función de activación actual (por ejemplo, la derivada de la ReLU):

$$\Delta^l = ((W^{l+1})^T \Delta^{l+1}) \odot (g^l)'(z^l)$$

- **Cálculo de gradientes locales:** Con los Deltas conocidos, obtenemos los gradientes exactos para este ejemplo específico multiplicando los vectores: Gradiente de Pesos: $\frac{\partial C}{\partial W^l} = \Delta^l (a^{l-1})^T$ Gradiente de Sesgos: $\frac{\partial C}{\partial W_0^l} = \Delta^l$

4. Descenso Estocástico por el Gradiente (Minilotes)

El descenso estocástico no actualiza los parámetros tras cada ejemplo individual, sino que procesa **minilotes** aleatorios de tamaño m para hacer estimaciones estadísticamente estables del error.

Para ajustar los parámetros finales de la red, se suman las matrices de gradientes calculadas en el Paso 3 para todos los m ejemplos del minilote, y se restan a los parámetros originales multiplicados por una tasa o factor de aprendizaje η :

- **Actualización de los Pesos:**

$$W^l \leftarrow W^l - \frac{\eta}{m} \sum (\Delta^l (a^{l-1})^T)$$

- **Actualización de los Sesgos:**

$$W_0^l \leftarrow W_0^l - \frac{\eta}{m} \sum \Delta^l$$